Why Paying for Storage Beats Free Networking in Cloud Bursting

Itamar Gefen
Technion

Aviad Zuck
Technion

Daniel Bransky
Technion

Moshik Hershcovitch
IBM Research

Danny Harnik
IBM Research

Dan Tsafrir
Technion

ABSTRACT

Hybrid cloud applications elastically burst to public clouds from an on-premise private cloud. In this setup only the public cloud directly charges applications for storing and processing data while the on-premise storage and network are already paid for and hence are considered free of charge. Consequently, application designers are naturally inclined to store and serve data remotely, while only paying for compute in the public cloud.

In this work we claim, perhaps counter-intuitively, that it is often the case that hybrid cloud applications should pay for cloud storage that is co-located with the cloud computations as a mean to reduce overall costs. A co-located cloud storage can serve as a cache for frequently-accessed data and help minimize data transfers from the on-premise cloud over relatively slow inter-cloud networks. This practice can implicitly reduce the public cloud compute charges by reducing wait time of remote reads from the on-premise location, which more than compensates for the increase in cloud storage costs. We demonstrate the potential benefits of our proposed caching scheme for two elastic computing models, on-demand and serverless, and multiple cache policies that utilize a cloud-resident object storage service. Our evaluation, based on cloud object store traces, suggests that this method can achieve substantial cost reduction, at times reducing costs by up to 85%.

CCS CONCEPTS

• **Information systems** → **Cloud based storage**; **Hierarchical storage management**.

KEYWORDS

Hybrid cloud, Cloud bursting, Caching, Serverless computing

This work is licensed under a Creative Commons Attribution International 4.0 License.
17th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage '25), July 10–11, 2025, Boston, MA, USA, 2025.
©2025 Copyright held by the author(s).
ACM ISBN 979-8-4007-1947-9/2025/07
https://doi.org/10.1145/3736548.3737826

1 INTRODUCTION

When working in a multi-site cloud setting, networking costs associated with moving data around often play a significant role since the pricing models of public clouds put a premium on *egress* network traffic, as opposed to relatively cheap cloud storage prices offered by all major cloud providers [1–5]. Consequently, when computation is run in a specific cloud with data that resides in a different cloud (or even a remote location) the network costs may become significant. Indeed, various works [6, 7] evaluate caching strategies aimed at reducing the network cost overheads.

In this work, we argue that caching in local cloud storage can yield significant *cost* savings even when the network is *free* of any charge. Specifically, we focus on a common relevant use case – *cloud bursting* as part of hybrid clouds. In cloud bursting data resides on-premise in a private cloud. Due to constraints such as lack of sufficient hardware, the application occasionally, and temporarily, elastically scales and moves some computation to public cloud instances (e.g., AWS) while data is still served remotely from the private cloud. Notably, on-premise storage and network are already paid for, and thus only the public cloud directly charges applications for storing and processing data. Applications running in such hybrid clouds are not directly charged for transferring data from their private on-premise cloud to the public cloud, since public cloud providers do not incur charges on *ingress* traffic. As such, there is a financial incentive to keep the data on-premise as reading it remotely incurs no additional explicit costs. Indeed, from internal discussions at IBM Cloud we learned that this is a common practice.

We observe that since public cloud compute time is billed also during waiting times for data being read from the on-premise site, there still exist both performance and *cost* benefits for caching data in the cloud, even though data transfer from on-premise to the cloud is seemingly free. Crucially, public cloud costs are dominated by the cost of compute time (See Fig. 2). By using a cloud storage cache in the same public cloud zone as the computation, an application can trade long network overheads from the client's on-premise cloud for high performance *intra-cloud* network and storage, e.g. 50ms

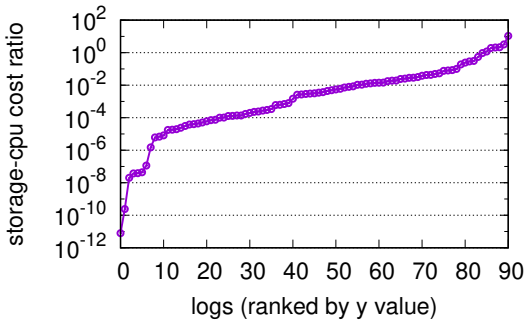

Figure 1: *Ratio of S3 storage cost for all objects over the CPU cost of a single AWS m5.large instance.*

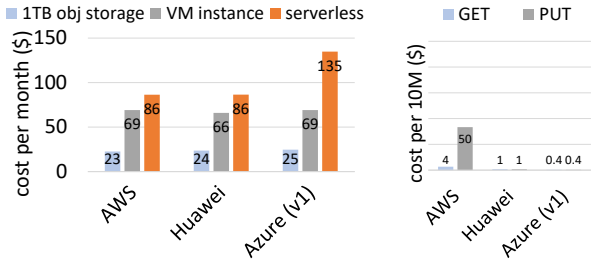

Figure 2: *Compute and object storage costs for three major cloud services.*

versus 10ms time-to-first-byte (TTFB). The resulting reduction in wait times can reduce the overall compute time and costs, which often outweigh the added local cache storage costs. To illustrate the potential for cost savings Figure 1 shows the ratio between the weekly costs of object storage and a general-purpose compute instance in Amazon AWS for a collection of cloud object store access logs [8, 9]. The results show that for 87% of the logs the overall data storage costs are $\leq$ 10% of compute costs.

The benefits of local caching for cloud bursting critically hinge on the instance type, specific workload at hand, cloud pricing models, and cache policy being used. We investigate the benefits of local caching for several potential cache policies in on-demand compute setups (§3), e.g. AWS EC2 [10]. Our analysis shows that permissively caching all data is beneficial in most cases, but also significantly increases costs for some logs. We therefore continue to explore more advanced caching policies, including a filter-based policy according to minimal number of accesses, as well as a time-to-live based policy. We then continue to explore the benefits of caching for serverless functions (§4), e.g., AWS Lambda [11], using cost models drawn from multiple major cloud providers. Our results, based on simulations of real-world workload traces, show that our caching solution can reduce overall costs by up to ~85%, with a median of ~60%, while capping cost overhead by 5% at most. Finally, we discuss potential extensions of our proposed cache solution as future work (§5).

Related Work. Caching in cloud has been widely explored, including various caching methods and media such as memory-based caching [12, 13], local disks, and cloud object storage. Like most caching solutions, cloud caching mainly focuses on achieving high hit rates, improving application speed, timely responsiveness and meeting SLO requirements. Additionally, caches are employed to re-use previously computed data [14–16]. Specifically for serverless computing, caching supports maintaining state or communicating between functions, rather than relying on persistent storage [17–19].

Multiple works use caching as a mean to reduce cloud costs with a focus on reducing the high networking costs between clouds and between cloud sites, including [6, 7, 20, 21]. Our work focuses on reducing cloud costs in setting where the network is free whereas the cache comes at a price.

2 MOTIVATION

Cloud bursting compute processing involves several factors which determine duration and costs. First, compute resources are billed at a fixed rate per usage time. Second, providers charge for the storage service utilized by the compute resource. Third, outgoing traffic is further charged. To wit, an on-demand m5.large AWS EC2 VM costs $0.096 per compute hour, $0.08/GB per month for a gp3 EBS block-based storage, $0.023/GB per month for S3 standard object storage, and $0.05-0.09/GB for outgoing traffic [1]. Notably, most providers do not impose costs on same-region data transfers from compute instances and object storage. Figure 2 illustrates cloud services costs of three major providers [1, 3, 22] for a general-purpose compute instance (2 CPUs, 8GB memory) and a 2GB serverless function.

The opportunity pointed by this work is that although ingress traffic to the public cloud during bursting is free, it still incurs significant inter-cloud communication overheads. These overheads vary according to exact setup and geographic distance between the public cloud to the on premise site, and typically range between 20-100ms of TTFB and 10-1000MB/s network throughput [23, 24]. We hypothesize that clients can reduce their overall cloud cost by trading slower inter-cloud traffic network overheads, and its related impact on extended compute time costs, for intra-cloud caches with lower access times, e.g., 10ms TTFB and 1GB/s throughput.

3 REDUCING COSTS BY CACHING

In this section we explore the potential of our proposed cache-based solution to reduce the duration and costs of burst compute on-demand instances, e.g., AWS EC2 [10].

3.1 Methodology

3.1.1 Traces. For our analysis, we use workload traces from the SNIA IBM Cloud Object Store repository [8], which log

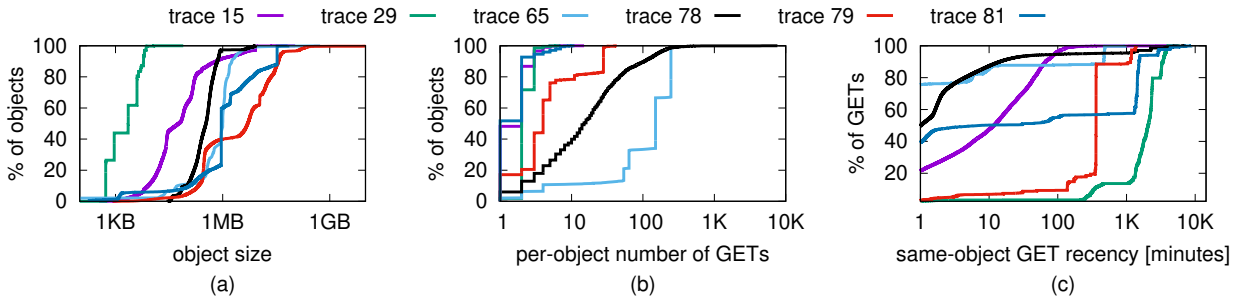

Figure 3: *Characteristics of six representative traces: CDFs of object sizes (a), object access frequencies (b), and same-object GET recency (c).*

one week of object storage REST accesses for client applications in a single IBM cloud region. To simplify, we only consider object GET requests and assume that the accessed objects are immutable. This assumption avoids potential consistency issues that could arise from caching mutable objects, which we leave for future work. Out of the 98 available traces, we exclude 6 that contain no object GET requests, as they are not pertinent to our analysis. We further exclude one particularly long and intensive trace to accelerate evaluation.

The IBM traces exhibit diverse access patterns across multiple dimensions, including object sizes, access frequency, and recency (i.e. time between consecutive reads of the same object), as discussed by Liu et al. [7]. Figure 3 summarizes these characteristics for six representative traces.

3.1.2 Simulation. We constructed an event-driven simulator (960 LoC) that processes the traces as object requests from a client cloud application on a bursting cloud instance. For each object access we simulate configurable access overheads for retrieving data from the client's on-premise cloud instance versus caching and serving it from a resident storage cache (i.e., same cloud region object storage).

Fig. 2 illustrates the main parameters used in our analysis for three major cloud providers. We further assume 50ms object TTFB latency and 100MB/s network bandwidth between on-premise and remote clouds, and 10ms TTFB and 1GB/s bandwidth between a bursting instance and its relevant cloud object storage service.

In 3.2 and 3.3 we establish a baseline for on-demand instances which run applications that are I/O-bound by scaling down traces and removing inter-request idle times (see §4 for analysis of more compute-intensive applications). For simplicity, we do not include per-request charges since they vary significantly between cloud providers (See Fig. 2). In §4 we extend our analysis to also include request costs.

3.2 Cache Policy Selection

In this work we consider caching using cloud object storage, which has pay-per-use pricing and is essentially unlimited in size. Classic caching methods on the other hand (e.g. LRU or ARC) evict items when the cache size is exhausted. With

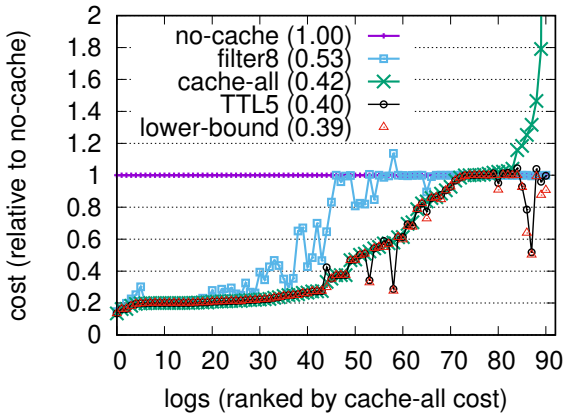

Figure 4: *Cost savings for each cache policy relative to the baseline without caching. In brackets we indicate the geometric mean of cost savings for each policy.*

object storage we can potentially cache *all* data. However, such permissive caching would naturally increase storage costs. As such, we further consider caching methods that separately decide whether to cache or evict for each object.

Fig. 4 illustrates our simulation results for potential caching policies, with geometric mean of cost savings for each policy in brackets. The `no-cache` policy (purple) establishes a baseline cost for each simulated trace, which always fetches objects remotely from the client's on-premise cloud. Other cost results are normalized to `no-cache` in order to establish their costs savings. A `lower-bound` (red) baseline considers only compute costs when objects are always cached in the bursting cloud, and thus serves as theoretical, not tightly-bound, optimal limit of cost savings for cache-based policies.

The `cache-all` policy (green) caches all objects after their first retrieval until the burst ends. The results, ordered by `cache-all`, illustrate that for on-demand bursting instances `cache-all` is beneficial for over 80% of logs and incurs negligible costs for 12% of logs . For 8% of logs `cache-all` significantly increases costs by over 5%, and by over 590% for one log (omitted for brevity), due to infrequent object re-use.

Since `cache-all` is highly permissive, we explored an approach to limit excessive caching using a filter-based admission policy [25], which caches an object only after a

pre-configured, per-object, access threshold is reached. This policy rewards workloads characterized by extensive object re-use, without incurring overheads for objects with no re-use. We experimentally determined the threshold to be eight. With this threshold the `filter8` policy (blue) improves costs for logs for which `cache-all` is detrimental, but reduces the benefits for ~60% of the logs in which the access frequency of objects is lower than the filter threshold.

Another caching strategy we evaluated is a Time-To-Live (TTL) policy, an approach originating from DNS caches, which evicts an object from the cache after a pre-configured period without accesses. Specifically, our $\texttt{TTL}_\texttt{i}$ policy assigns each object a TTL that caps its storage cost at i% of the compute cost of fetching it remotely. For example, for `TTL100` the storage and compute costs are identical.

More formally, we define $C_{compute}(obj)$ as the compute cost of fetching the object from on-premise, which is the sum of compute time cost until the object retrieval starts (i.e., TTFB) and transfer time, which is a function of the object size and relevant network bandwidth. $C_{cache}(obj, t)$ is defined as the charges for redundantly caching an object for duration t. These charges include the cost of compute time and related storage request costs (null in this section), i.e., a single cache PUT and DELETE when the TTL expires.

We then find t^*, which is the maximal time for which $C_{cache}(obj, t^*) < \frac{i}{100} \cdot C_{compute}(obj)$, and assign this value as the object's TTL. Small TTL values mean that cost overhead is capped at a lower point. On the other hand, for objects with long periods of time between accesses small TTLs can lead to cache misses.

We experimentally determined `TTL5` (black) as our preferred $\texttt{TTL}_\texttt{i}$ policy (See §3.3). The `TTL5` policy reduces overall costs for 93% of the logs, with a median cost saving of 65%. For six logs `TTL5` increases costs, but as expected, caps the cost increase by 5% as determined by our definition of $\texttt{TTL}_\texttt{i}$.

3.3 TTL Sensitivity Investigation

Figure 5 illustrates our exploration of the $\texttt{TTL}_\texttt{i}$ policy. The figure shows logs by percentiles of cost relative to the `no-cache` baseline for representative i values for $\texttt{TTL}_\texttt{i}$. The results show that for the majority of logs (90%) most TTL values result in similar gains, close to `lower-bound`. This result is due to the relatively short intervals between repeated object accesses, characterizing most of our traces. For the rest of the logs, cost increases relative to `no-cache` are capped by the TTL value. However, for some logs `TTL1` occasionally incurs more costs than other TTL values, while `TTL20` increasingly underperforms for the 90th and 99th percentiles.

To further determine the most preferred $\texttt{TTL}_\texttt{i}$ policy, we focus on workloads that are incompatible for `cache-all`, either because they do not exhibit object re-use or because their

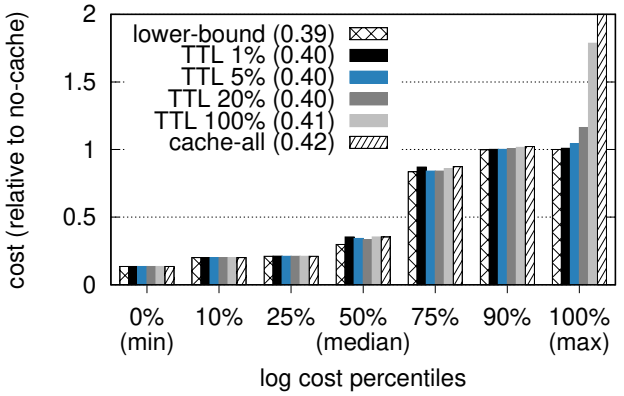

Figure 5: *Log cost percentiles relative to* `no-cache` *for TTL-based caching with varying thresholds.*

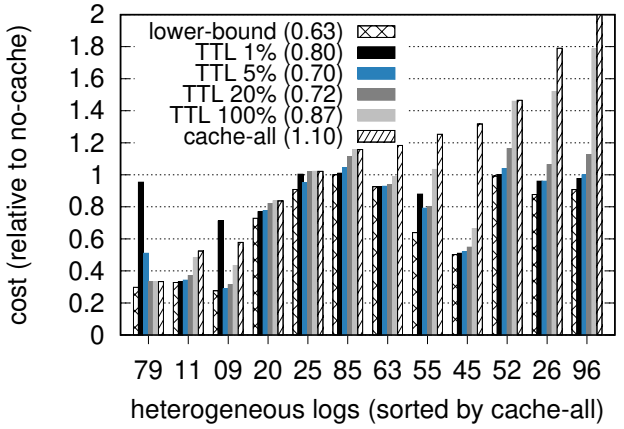

Figure 6: *Cache results for heterogeneous workloads for which* `cache-all` *leaves room for improvement.*

storage cost accumulates more than necessary. Specifically, we focus on workloads with least 10% room for improvement, i.e. `lower-bound` $< 0.9 \times$ `cache-all`. The results in Fig. 6 show twelve workloads that meet this criteria and focuses on their compatibility for different TTL values. For these workloads, lower TTL values are generally preferable since they are the safest with respect to not exceeding `no-cache`. However, too small TTL values (e.g. `TTL1`) again do not yield sufficient cost savings for traces with relatively long durations between object re-uses. We therefore conservatively conclude that `TTL5` is the preferable TTL policy.

3.4 Factoring Per-request Costs

In the simulation above we ignored the effect of per-request costs, e.g. PUT and GET. Request costs are incurred by most cloud object service providers but highly vary between vendors (See Fig. 2). For providers with small request costs, e.g., Azure, the effect on overall costs is minor. However, if these costs are high, e.g. AWS, they must be considered in the caching policy. To incorporate these costs we revised our TTL policy so that if request costs exceed $\frac{i}{100} \cdot C_{compute}(obj)$

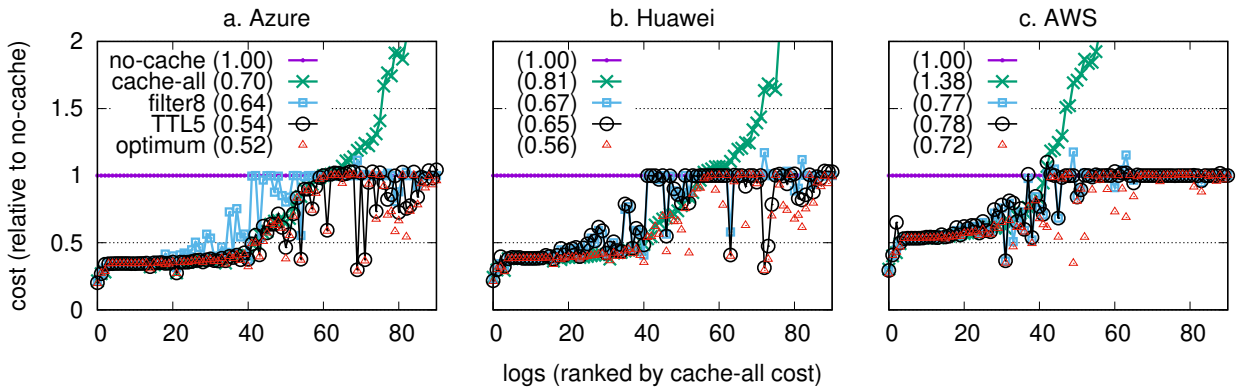

Figure 7: *Costs relative to* `no-cache` *for the cost models of three major cloud services, including request costs.*

we fall back to `filter8`. We found that in such cases, filtering on admission is a better policy. The results in the next section incorporate request costs.

4 THE CASE OF SERVERLESS COMPUTING

In order to simulate applications that lie in the intermediate region of the continuum between I/O and CPU bound applications we utilize the properties of the popular serverless computing service [11, 22, 26] (e.g., AWS Lambda).

When running the IBM traces on our simulator for on-demand instances with original object access timings, the cloud instance run time and compute cost are determined by the period between the first and last operation. On the other hand, with serverless functions, application compute operations occur only when strictly necessary, and costs are incurred only for active durations. Therefore, we now use the original traces to incur costs for serverless functions during object retrieval operations.

Specifically, we consider a model where object retrievals are followed by subsequent compute periods in varying lengths. We use data from a recent analysis of Huawei's FunctionGraph serverless service which showed that the median function run time is 10ms and uses a serverless instance with 2GB of memory [27]. We therefore extend our traces to further include a corresponding 10ms compute time for each object retrieval request and repeat our simulated trace runs for our previously proposed cache policies. We assume that the client maintains a persistent local cache beyond serverless function invocations, i.e. in the same cloud region as the serverless instance, by paying for an object storage service.

4.1 Serverless results

Fig. 7 illustrates results for cost models of three major cloud providers with differing request costs. The results illustrate that for providers with high request costs (e.g., AWS), our

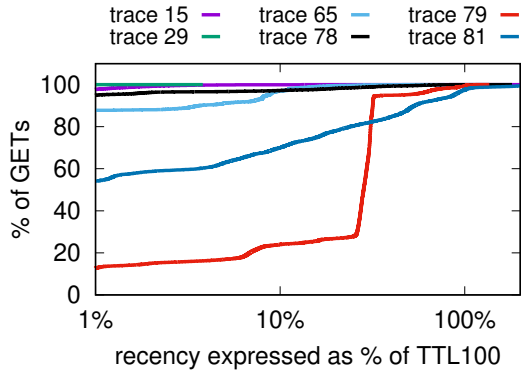

Figure 8: *CDF of same-object GET recency as a percentage of* `TTL100`*, using Azure v1 pricing, for six representative traces.*

TTL-based policy mostly degenerates to `filter8`. For providers with low request costs, e.g Azure, the `cache-all` policy is disadvantageous for more traces than for on-demand setups (Fig. 4), mostly due to the disparity between the extended caching period relative to the serverless compute time. However, our TTL-based policy still benefits 82% of logs under these conditions. This result stems from the fact that subsequent object accesses are typically much shorter than their corresponding `TTL100` values, as illustrated in Figure 8.

Next, we investigate the utility of caching by gradually increasing the CPU time of serverless functions starting from 1ms, which is effectively I/O-bound, up to 1000ms, the maximum function duration observed in [27]. Figure 9 shows the geometric mean and distribution of the relative cost for our `TTL5` policy for increasing function CPU durations in different clouds. The results show diminishing cost savings as function duration increases, highlighting a reduced benefit from caching for longer-running (more CPU-intensive) functions. Similarly, the cost increases of the worst-performing traces also taper off and become negligible at higher durations. Nevertheless, caching still yields substantial benefits even for CPU-intensive functions. With an additional 10ms of CPU time following an object read, we observe cost savings

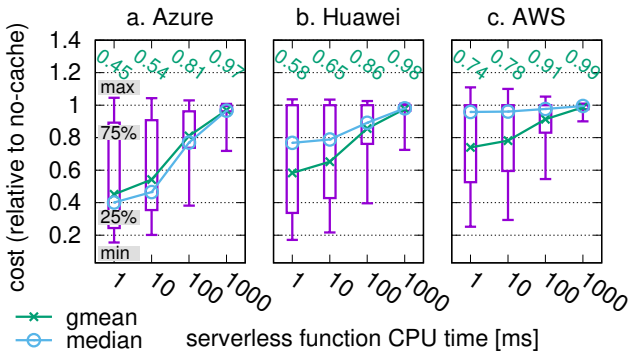

Figure 9: *The effect of extended serverless function CPU time on costs, relative to* no-cache*, for cost models of three major cloud services.*

of up to 80%, with a geometric mean of 54%. For additional CPU times of 100ms and 1000ms, the maximum observed cost savings are 62% and 27%, with corresponding geometric means of 19% and 3%, respectively.

Notably, in our analysis we assume no overlap between the serverless function CPU processing and IO times, so that together they compose the overall function runtime, i.e. $runtime = CPU_{time} + IO_{time}$. When CPU_{time} and IO_{time} overlap, then the savings from caching increase as long as the overlap is small relative to the saved CPU time. This is the common case in our analysis, since serving data remotely is significantly slower than doing so locally (by at least 40ms in our analysis) while the median function run time is 10ms.

5 FUTURE WORK

Adaptiveness and Prefetching. The caching policies previously discussed are static and do not adapt to client-specific access patterns. While this approach is simpler and avoids potential issues arising from shifting access patterns, adaptive policies may offer additional benefits. In particular, adaptive strategies might enable prefetching of objects prior to their first access, thereby reducing the cost associated with first-time reads.

Additional Storage Services. So far we only considered object storage (e.g., AWS S3) as our local cache backend. However, cloud providers offer many types of storage services varying by performance, pricing, memory type, and durability [1, 3, 28], which may also be used for local caches. For brevity, we focus on costs from AWS services [12, 29, 30].

Figure 10 compares cost savings for our TTL5 policy in a serverless setup with various storage services used for caching. S3 object storage (black) is the same object storage we considered so far. S3 Express One Zone (green) is a high-performance object storage, designed for latency-sensitive applications, and ElastiCache (gold) is AWS's fully managed in-memory storage. The results illustrate that using faster memories may be beneficial, often yielding compute time

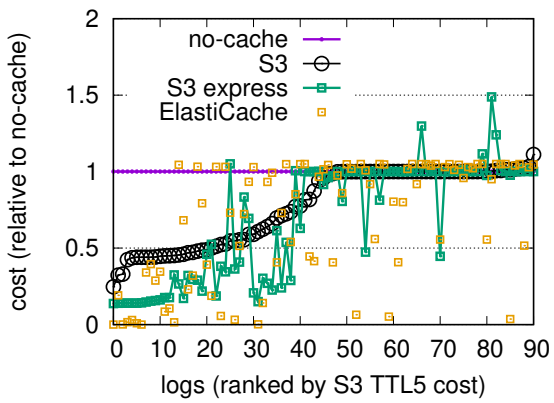

Figure 10: *Costs of serverless applications with different types of AWS memory services using a* TTL5*-based cache, relative to* no-cache.

savings that more than offset their much higher storage costs. However, the results show that paying more for faster types of cloud storage is not directly correlated to cost benefits. This result is due to the fact that different memory types fit different types of object sizes and access frequencies, indicating the potential for employing a heterogeneous tiered cache with multiple types of storage for caching client data.

We will rigorously explore the applicability of our proposed cache scheme for different paid storage services, as well as extending our solution to a tiered cache which adaptively, and dynamically, moves objects between different tiers. We hypothesize that the benefits of such multi-layered caches are not limited to the cloud bursting scenario; rather, they are also beneficial for reducing costs of general VM based and serverless cloud computing workloads.

Batching. In §4 we showed that high request costs can significantly reduce the utility of cloud-resident object-based caches. One possible remedy for this limitation is to reduce the number of expensive cache PUT requests by batching together cache updates into one "container" object. The trade-off is that object GET requests must now access ranges within a larger object, which requires maintaining an additional indirection layer in memory. Furthermore, since ranged object updates are not supported, modifying an object now requires updating its entire container. We will investigate whether this optimization yields cost benefits without significantly hampering performance and data durability.

ACKNOWLEDGMENTS

This study was partially supported by a grant from IBM titled "Technion – IBM Research Collaboration."

REFERENCES

[1] AWS. Amazon pricing. https://aws.amazon.com/pricing. Accessed: March 2025.

[2] Huawei. Cloud object storage service. https://support.huaweicloud.com/intl/en-us/obs/index.html. Accessed: March 2025.

[3] Microsoft. Azure pricing. https://azure.microsoft.com/en-us/pricing. Accessed: March 2025.

[4] Google. Google cloud storage. https://cloud.google.com/storage. Accessed: March 2025.

[5] IBM. Ibm cloud object storage. https://www.ibm.com/products/cloud-object-storage. Accessed: March 2025.

[6] Zhe Wu, Michael Butkiewicz, Dorian Perkins, Ethan Katz-Bassett, and Harsha V Madhyastha. Spanstore: Cost-effective geo-replicated storage spanning multiple cloud services. In *SOSP'13: Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 292–308, 2013.

[7] Shu Liu, Xiangxi Mo, Moshik Hershcovitch, Henric Zhang, Audrey Cheng, Guy Girmonsky, Gil Vernik, Michael Factor, Tiemo Bang, Soujanya Ponnapalli, et al. Skystore: Cost-optimized object storage across regions and clouds. *arXiv preprint arXiv:2502.20818*, 2025.

[8] SNIA. I/o trace data files. https://iotta.snia.org/traces/. Accessed: March 2025.

[9] Ohad Eytan, Danny Harnik, Effi Ofer, Roy Friedman, and Ronen I. Kat. It's Time to Revisit LRU vs. FIFO. In *HotStorage'20: 12th USENIX Conference on Hot Topics in Storage and File Systems*, 2020.

[10] AWS. Compute service - amazon ec2. https://aws.amazon.com/pm/ec2. Accessed: April 2025.

[11] AWS. Amazon lambda. https://aws.amazon.com/pm/lambda/. Accessed: March 2025.

[12] AWS. Amazon elasticache. https://aws.amazon.com/elasticache/. Accessed: March 2025.

[13] Google. Google memorystore. https://cloud.google.com/memorystore. Accessed: March 2025.

[14] Nicolas Le Scouarnec, Christoph Neumann, and Gilles Straub. Cache Policies for Cloud-Based Systems: To Keep or Not to Keep . In *2014 IEEE 7th International Conference on Cloud Computing (CLOUD)*, pages 1–8, July 2014.

[15] Trausti Saemundsson, Hjortur Bjornsson, Gregory Chockler, and Ymir Vigfusson. Dynamic performance profiling of cloud caches. In *SOCC'14: Proceedings of the ACM Symposium on Cloud Computing*, pages 1–14, 2014.

[16] Damiano Carra, Giovanni Neglia, and Pietro Michiardi. Ttl-based cloud caches. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, pages 685–693, 2019.

[17] Djob Mvondo, Mathieu Bacou, Kevin Nguetchouang, Lucien Ngale, Stéphane Pouget, Josiane Kouam, Renaud Lachaize, Jinho Hwang, Tim Wood, Daniel Hagimont, et al. Ofc: an opportunistic caching system for faas platforms. In *EuroSys'21: Proceedings of the Sixteenth European Conference on Computer Systems*, pages 228–244, 2021.

[18] Ao Wang, Jingyuan Zhang, Xiaolong Ma, Ali Anwar, Lukas Rupprecht, Dimitrios Skourtis, Vasily Tarasov, Feng Yan, and Yue Cheng. {InfiniCache}: exploiting ephemeral serverless functions to build a {cost-effective} memory cache. In *FAST'20: 18th USENIX conference on file and storage technologies*, pages 267–281, 2020.

[19] Francisco Romero, Gohar Irfan Chaudhry, Íñigo Goiri, Pragna Gopa, Paul Batum, Neeraja J Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. Faa$t: A transparent auto-scaling cache for serverless applications. In *SoCC'21: Proceedings of the ACM symposium on cloud computing*, pages 122–137, 2021.

[20] Binbing Hou and Feng Chen. Gds-lc: A latency- and cost-aware client caching scheme for cloud storage. *ACM Trans. Storage*, 13(4), November 2017.

[21] Hojin Park, Ziyue Qiu, Gregory R Ganger, and George Amvrosiadis. Reducing cross-cloud/region costs with the auto-configuring macaron cache. In *SOSP'24: Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 347–368, 2024.

[22] Huwaei. Functiongraph. https://www.huaweicloud.com/eu/product/functiongraph.html. Accessed: March 2025.

[23] Kentick. Kentick cloud latency map. https://clm.kentik.com/?q=505de2a13d5. Accessed: March 2025.

[24] AWS. Aws direct connect. https://aws.amazon.com/directconnect/pricing/. Accessed: March 2025.

[25] Elizabeth J. O'Neil, Patrick E. O'Neil, and Gerhard Weikum. The lru-k page replacement algorithm for database disk buffering. In *SIGMOD'93: Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pages 297–306, 1993.

[26] Microsoft. Azure serverless. https://azure.microsoft.com/en-us/solutions/serverless. Accessed: March 2025.

[27] Artjom Joosen, Ahmed Hassan, Martin Asenov, Rajkarn Singh, Luke Darlow, Jianfeng Wang, and Adam Barker. How does it function? characterizing long-term trends in production serverless workloads. In *SoCC'23: Proceedings of the 2023 ACM Symposium on Cloud Computing*, pages 443–458, 2023.

[28] Google. Google cloud pricing. https://cloud.google.com/pricing. Accessed: March 2025.

[29] AWS. Amazon s3 express one zone storage class. https://aws.amazon.com/s3/storage-classes/express-one-zone/. Accessed: March 2025.

[30] AWS. Amazon s3. https://aws.amazon.com/s3/. Accessed: March 2025.